

Progetto di Ingegneria Informatica
Prof. Marco Domenico Santambrogio, Ph.D. Student Mirko Coggi

Dipartimento di Elettronica, Informazione e Bioingegneria
Politecnico di Milano

**Accelerazione dell'allineamento di sequenze
a grafi genomici su GPU**



POLITECNICO
MILANO 1863

Gabriele Amodeo [10829738]
Leonaro Tisato [10844472]

May 2025

Accelerating Partial Order Alignment Algorithm For Genome Alignment Task On Heterogenous Architectures (CPUs and GPUs)

Gabriele Amodeo, Leonardo Tisato

Dipartimento di Elettronica, Informazione e Bioingegneria

Politecnico di Milano

Milan, Italy

{gabriele1.amodeo, leonardo.tisato}@mail.polimi.it

Abstract—Graph-based representations of pangenomes provide a more comprehensive model of genetic variation compared to conventional linear references. Yet, their widespread adoption remains significantly constrained by the substantial computational demands of sequence-to-graph alignment operations. In this research, we present a sophisticated GPU-accelerated alignment tool constructed upon an enhanced CONSENT-GPU computational kernel, methodically extended to support the complete semantic richness of variation graphs—encompassing complex branching pathways, insertions, and deletions — while simultaneously exploiting parallelization opportunities both across multiple sequencing reads and within each alignment process. Our implementation effectively leverages the computational capabilities of contemporary graphics processing architectures by reorganizing graph structures into memory-efficient representations, systematic rearrangement of read batches to optimize hardware utilization, and wise use of GPU-shared memory resources. When tested on synthetic benchmarks that mimic realistic sequencing errors and complex graph topologies, our GPU-based aligner matches the accuracy of exhaustive CPU implementations, yet processes data faster than V-ALIGN. This shows that, due to optimized kernel design and efficient data organization, GPU-accelerated sequence-to-graph alignment can scale to the computational demands of large-scale pangenomic studies.

I. INTRODUCTION

Genomic research has entered an unprecedented era of data generation, driven by sequencing techniques that systematically profile thousands of individuals, diverse populations, and intricate microbial communities. Within this rapidly evolving landscape, the concept of the pangenome—representing the complete collection of genes and regulatory elements across all variants of a species—has emerged as an essential framework for comprehensively capturing both the conserved “core” genome and its multitude of natural variations. Traditional bioinformatics workflows have historically depended upon single linear reference sequences, attempting to adapt existing alignment and variant-calling methodologies to accommodate ever-expanding datasets. However, this standard approach overlooks much of the genetic variation present across populations.: particularly in genome-wide association

studies (GWAS), where the well-documented phenomenon of reference allele bias systematically distorts results by undercalling or misaligning reads that diverge from the arbitrarily chosen linear reference, ultimately leading to significantly distorted variant frequency estimates and critically missed genetic discoveries. A biologically more authentic representation of population-level variation emerges through modelling reference cohorts as sophisticated variation graphs. In these graphs, nodes store shared sequence regions, while directed edges capture structural changes like insertions and deletions. By accommodating multiple traversal paths through the graph, this advanced framework inherently incorporates both inter-individual variation (differences between individuals) and intra-individual variation (differences within a single genome), substantially reducing mapping bias and dramatically enhancing sensitivity to rare or population-specific allelic variants. According to the *Human Pangenome Reference Consortium* (HPRC) [8], pangenome graphs offer a significantly more complete and accurate view of genetic diversity compared to linear assemblies. While pioneering graph-based alignment systems already demonstrate measurable improvements in alignment accuracy compared to traditional linear methods, these accuracy gains come at the expense of a substantial—often prohibitive—increase in computational requirements. Aligning sequencing reads to graph-based references introduces fundamental algorithmic challenges that transcend conventional linear alignment. The computational search space expands exponentially as reads must potentially follow multiple branching paths, necessitating sophisticated backtracking mechanisms and complex nondeterministic path-scoring algorithms. Modern alignment tools try to handle this complexity using approximation techniques such as heuristic simplifications, alignment banding, and pruning of the search space. However, these shortcuts often trade alignment sensitivity for speed and struggle with high-coverage datasets or long-read sequencing data. Furthermore, the inherent diversity of graph topological structures and the persistent absence of standardized file formats significantly impede cross-platform interoperability and consistent optimization across different software implementations. Hardware acceleration leveraging

graphics processing units (GPUs) has fundamentally transformed classical sequence-to-sequence alignment by harnessing thousands of parallel computational cores to deliver performance improvements measured in orders of magnitude. However, extending these acceleration benefits to sequence-to-graph alignment presents unique challenges: irregular memory access patterns, highly dynamic control flow requirements, and the necessity to efficiently traverse complex data structures all create substantial obstacles to straightforward GPU implementation. While the earliest GPU-based graph alignment system successfully demonstrated the basic feasibility of offloading graph alignment operations to hardware accelerators, it remains fundamentally limited to parallelization across multiple reads and lacks comprehensive support for advanced graph-specific features. To effectively overcome these computational bottlenecks and fully realize the potential of GPU acceleration for graph-based alignment, entirely new computational kernels and meticulously optimized data representation layouts are required—forming the central motivation and primary research focus of our present work.

II. AIM

In this work, we aim to address the computational challenges of sequence-to-graph alignment by using modern GPU hardware. Our goal is to turn sequence-to-graph alignment from a computational bottleneck into an efficient parallel process on GPUs. Building on the CONSENT-GPU [1] kernel, we used its GPU alignment infrastructure to implement a different strategy: aligning sequencing reads directly to a fixed variation graph, rather than performing graph-to-graph alignment. While CONSENT-GPU transforms each read into a graph and performs graph-to-graph alignment followed by merging, our approach aligns batches of sequences directly to a fixed reference graph provided as input. We will work on two levels of parallelism: enabling each alignment to progress simultaneously across multiple parts of the graph, while also processing thousands of different sequences at the same time. We will use GPU programming techniques to reduce processing delays and improve memory access efficiency. Our implementation takes as input the widely used GFA (Graphical Fragment Assembly) file format, which represents genome graphs as collections of segments, links, and paths. We have developed a custom preprocessing tool that efficiently converts these GFA files into our specialized GPU-optimized data structures. This preprocessing step flattens the complex graph representation into memory-efficient arrays that enable fast traversal on GPU hardware. Our tool handles various graph complexities while preserving the full semantics of the original graph structure. The preprocessing step is crucial for our approach, as it transforms the graph from a representation optimized for construction and visualization into one optimized for massively parallel computation. This includes creating auxiliary data structures that encode path relationships and graph topology in ways that minimize thread divergence during GPU execution. By developing this custom GFA conversion tool, we ensure that our aligner can work

directly with standard graph formats used by the genomics community, enhancing its practical utility in real-world bioinformatics pipelines. We will evaluate our tool by comparing it with leading alignment tools, measuring overall processing speed, start-to-finish time, and memory usage across graph complexities and read characteristics. We will verify alignment accuracy using synthetic test cases that reflect realistic sequencing errors and graph structures and compare the results with those from a full CPU-based implementation. Our goal is to show that sequence-to-graph alignment can run faster on GPU hardware, speeding up the generation of accurate results, even when dealing with complex variation and large datasets.

III. RELATED WORKS - SOA

The first sequence-to-graph alignment systems prioritized algorithmic correctness and feature completeness rather than computational efficiency. PaSGAL [3] introduced one of the pioneering graph-based alignment frameworks, establishing important theoretical foundations for the field. However, despite its conceptual innovations, the software remains effectively non-functional in its publicly available implementation, limiting its practical utility. In contrast, V-ALIGN [4] delivered a working CPU-based alignment system that guarantees mathematically optimal, completely heuristic-free alignments on variation graphs. This approach achieves 100% accuracy, though at the considerable cost of dramatically increasing computational runtime and substantial memory requirements. Together, these early systems conclusively demonstrate that exhaustive graph alignment calculations are theoretically feasible on conventional computing architectures, while simultaneously highlighting the critical need for transformative performance improvements if graph-based methodologies are to achieve practical scalability for contemporary high-throughput sequencing datasets. Attempts to significantly accelerate graph alignment computation have thus far yielded limited results. HGA [2] represents the first substantive effort to offload graph alignment operations to GPU hardware, implementing a heterogeneous CPU-GPU computational pipeline that effectively parallelizes across multiple read sequences but crucially fails to exploit the rich parallelism potential within each alignment task. Furthermore, its implementation encounters significant difficulties when processing complex graph topological features and ultimately falls considerably short of production-ready stability and completeness. Previous research on partial-order alignment (POA) explored promising avenues for intra-graph parallelism by decomposing multiple sequence alignment problems into a series of graph merging operations. However, this architectural approach was specifically engineered for the particular constraints of the POA use case, converting all sequence relationships into directed acyclic graphs (DAGs) for consensus sequence construction, and consequently cannot be directly applied to general-purpose reference-graph alignment scenarios. As a result, the current technological landscape lacks any GPU-based solution that fully supports and effectively leverages both inter-sequence and intra-sequence parallelism when operating on general

variation graphs, thereby creating the compelling technical motivation for our extension of the CONSENT-GPU computational kernel to address this critical capability gap.

IV. IMPLEMENTATION

Our implementation is inspired by the GPU-accelerated partial order alignment strategy presented in CONSENT-GPU, where multiple sequences are transformed into partial order graphs and incrementally aligned through graph fusion. However, while CONSENT iteratively merges sequence graphs into a unified structure, our approach simplifies this by focusing solely on the alignment between a fixed reference graph and raw input sequences.

The GPU implementation is structured into distinct kernel modules that work in unison to optimize computational efficiency. Our architecture enables the alignment of multiple sequences in parallel while maintaining scalability across various batch sizes and sequence lengths. The following subsections detail the core components of this implementation.

S	seq1	A			
S	seq2	GC			
S	seq3	TA			
L	seq1	+	seq2	+	*
L	seq1	+	seq3	+	*
L	seq2	+	seq3	+	*

Letters:	A G C T A
Edges:	-1 0 1 0 2 3
Edge Bounds:	0 1 2 3 5 6

Fig. 1. Example of a graph conversion from GFA format to CSR representation.

A. Alignment Preparation

The number of blocks, the input reads in FASTA format, and the graph in GFA format are provided via the command line. The set of reads is partitioned into balanced batches based on the specified number of blocks and the total number of reads, ensuring an even distribution of the workload. The graph is then converted from the GFA format to a Compressed Sparse Row (CSR) representation, as shown in Figure 1, which facilitates the alignment computation logic and provides memory-efficient storage.

B. Task Batching and Parallelization

The implementation processes batches of MSA tasks to fully utilize GPU resources. Task batching involves segmenting the entire set of sequences into smaller batches, each of which is aligned independently.

The batch size is calculated based on the number of blocks provided and the total number of reads. Before beginning the alignment tasks, a check is performed to ensure memory limits are not exceeded.

C. Kernels Organization

The GPU implementation begins with the initialization of global memory on the device to allocate space for storing the input sequences, the reference graph and the DP-matrix. Each kernel processes a single sequence alignment task, with each GPU block managing one alignment independently. The pipeline consists of 2 key phases:

- *Dynamic Programming Matrix Initialization:* two kernels are employed during the initialization phase. The first kernel initializes the `d_offsets` array, which stores the offsets corresponding to the lengths of each anti diagonal in the dynamic programming (DP) matrix. The second kernel initializes the first row and column of the DP matrix by iteratively assigning insertion scores along the row and deletion scores along the column.
- *Dynamic Programming Matrix Computation:* a scoring kernel calculates the alignment scores by iteratively processing the anti-diagonals of the dynamic programming (DP) matrix in parallel. Each thread handles a single column of the current anti-diagonal, ensuring high parallelism as shown in Figure 2.

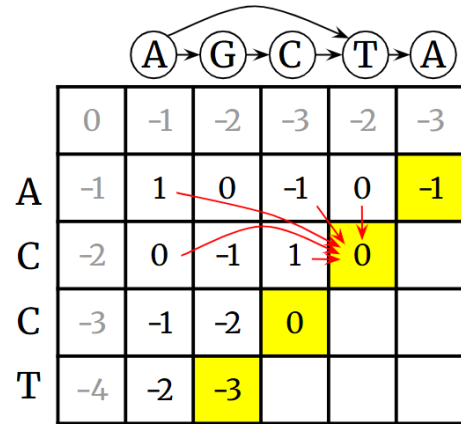


Fig. 2. Example of threads filling a dp-matrix anti-diagonal in parallel.

To handle dependencies within the DP matrix, the edges and predecessor nodes of each partial order graph are transferred to shared memory, reducing global memory access overhead. Threads within a block compute scores for each cell in parallel, leveraging the directed acyclic nature of partial order graphs.

D. Scoring Kernel Optimization

The scoring kernel uses three arrays to encode the partial ordered graph: the `dyn_letters_global` array for node bases, the `dyn_edges_global` array for the edges list, and the `dyn_edge_bounds_global` array for the edge offsets of each node. To minimize memory latency:

- Prior to the scoring phase, the kernel loads the relevant sequence data, graph edges, and structural offsets into shared memory to reduce global memory access latency. The graph structure is processed to generate auxiliary arrays (e.g., `lx_start`, `ly_start`, `left_x`, `left_y`)

TABLE I
EXECUTION TIMES AND MEMORY USAGE FOR DIFFERENT DATASETS

Read Dataset	Total Time (s)	Memcpy Time (s)	Align Time (s)	GCUPs	Global Memory (MB)	Shared Memory (B)
S1	1.179	0.268	0.164	3.05	101	2,301
S2	1.335	0.290	0.316	3.16	108	2,323
S3	4.233	0.726	2.691	3.71	360	2,334
L1	5.131	0.140	3.632	5.50	2,106	21,916
L2	10.505	0.169	9.070	5.51	2,133	21,905
L3	19.538	0.165	18.101	5.52	2,184	21,916

that encode the partial order dependencies of the sequences and allow efficient traversal during alignment.

- The DP matrix is logically stored using anti-diagonal offsets to facilitate coalesced memory access patterns, and alignment scores are computed for each cell by considering three transition paths: match/mismatch (diagonal), insertion (horizontal), and deletion (vertical). For each cell, the kernel evaluates the optimal score based on these transitions, taking into account the graph’s topology and predecessor dependencies.
- The highest scoring cell in the DP matrix is dynamically tracked across threads using warp-level reductions.

To further optimize and add information to the alignment result, two auxiliary matrices, `moves_x` and `moves_y`, are used for later backtracking. Once all anti-diagonals are processed, the traceback is performed to reconstruct the optimal alignment path, by a single thread as it contributes minimally to the overall runtime.

V. EVALUATION

All experiments were conducted on a consumer-grade NVIDIA GTX 1650 GPU with 4 GB of RAM.

We constructed synthetic datasets as follows: a reference sequence of fixed length was generated randomly and used to derive both the input reads and the corresponding reference graph. For each read, approximately 20 single-base mutations were introduced at random positions along the reference sequence. Each mutation consisted of either a deletion or an insertion of a randomly selected base. Table II reports the characteristics of the corresponding read sets.

The reads datasets used in our experiments consist of both short and long reads, identified by their set names. Sets whose names begin with S contain short reads, with each read approximately 100 base pairs in length. Conversely, sets labeled with L are composed of long reads, each around 1,000 base pairs. This distinction allows for evaluating the aligner’s performance across a range of read lengths.

The graph was built by introducing approximately 20 randomly distributed Single Nucleotide Polymorphisms (SNPs) into the original reference sequence. This ensures a topology with a controlled degree of variation and complexity while maintaining a comparable number of vertices across tests for fair evaluation. Table III summarizes the main properties of the generated graphs.

TABLE II
SYNTHETIC READS SETS USED FOR EVALUATION

Name	Number of reads	Number of bases
S1	50,000	5,000,228
S2	100,000	9,998,665
S3	1,000,000	100,000,701
L1	20,000	20,000,221
L2	50,000	49,998,769
L3	100,000	100,000,221

TABLE III
SUMMARY OF TEST GRAPHS

Graph	Number of vertices	Number of edges
Graph S	104	107
Graph L	1,071	1,091

Both short-read and long-read experiments were conducted using graphs of similar size and complexity, and the number of reads was varied to explore the scaling behavior of the aligner.

Therefore, it follows that in all experiments, short-read sets (those labeled with prefix *S*) were aligned against *Graph S*, while long-read sets (labeled with prefix *L*) were aligned against *Graph L*.

Note that the datasets used in this evaluation were designed to highlight the performance characteristics of our implementation, with a focus on controlled and relatively small-scale scenarios. Future work will include benchmarking on real-world data and exploring parameter tuning to better adapt each method to different application settings.

A. Results

Table I reports the execution time, throughput misured in Giga Cells Updated Per Second (GCUPs), and memory usage of our GPU-based aligner across short-read (S1–S3) and long-read (L1–L3) datasets. As expected, both total and alignment times scale with the number of reads and their lengths. Throughput improves with read length, reaching over 5.5 GCUPS for long-read sets, compared to 3–3.7 GCUPS for short reads.

Memory usage remains well within the 4 GB GPU limit. Global memory usage scales with dataset size, ranging from

TABLE IV
COMPARISON OF THE PERFORMANCE OF SEQUENCE-TO-GRAPH ALIGNERS (TIME IN SECONDS). TO INDICATES TIMEOUT, - - INDICATES NOT TESTED.

Reference Graph Read Dataset	S1	Graph S S2	S3	L1	Graph L L2	L3
Giraffe	- -	- -	- -	42.147	86.120	171.124
V-Align	47.860	95.190	968.038	TO	TO	TO
GWFA	92.275	314.623	TO	82.938	455.199	TO
Our Tool	1.179	1.335	4.233	5.131	10.505	19.538

101 MB (S1) to 2.1 GB (L3). Shared memory allocation increases significantly for long-read datasets, reflecting the larger associated intermediate data structure.

The memcopy time (host-device transfer) is consistently small relative to total runtime, confirming that GPU computation dominates the workload and that data transfer does not constitute a major bottleneck in the current implementation.

The tool exhibits an approximately linear growth in execution time with respect to the total alignment workload, defined as the number of reads multiplied by their average length. This is evident from the long-read sets (L1-L3), where the total time increases proportionally with the input size. For example, moving from L1 to L3 results in a nearly 4× increase in alignment time, consistent with the 5× increase in the number of reads.

The GCUPs metric also increases with small datasets and stabilizes with larger datasets, reflecting improved GPU utilization and amortization of overhead costs. Longer reads, in particular, allow for better saturation of GPU resources, which explains the higher throughput observed in long-read sets.

All alignment results produced by our implementation are exact, as the algorithm does not employ any heuristics or approximations. By computing the full dynamic programming matrix, it guarantees 100% accuracy across all test cases, offering a reliable baseline for evaluating more heuristic-based approaches.

B. Results Comparison

Table IV compares the execution time of our tool with other state-of-the-art sequence-to-graph aligners, including Giraffe [6], V-Align, and GWFA [5]. All tools were executed on the same datasets to ensure consistency and fairness in evaluation, using the GGBS benchmark suite [7].

Our GPU-based aligner significantly outperforms all tested tools in terms of runtime. Figures 3 and 4 illustrate the execution time of our aligner compared to existing tools across short- and long-read datasets. While the plots are not to scale and do not use a logarithmic or normalized axis, they are intended to visually highlight the substantial differences in runtime and the consistent speedup achieved by our implementation. The trends clearly show that our tool maintains lower execution times even as dataset size increases, both in terms of number of reads and read length.

For short-read datasets (S1-S3), our implementation is up to 80× faster than V-Align and over 70× faster than GWFA

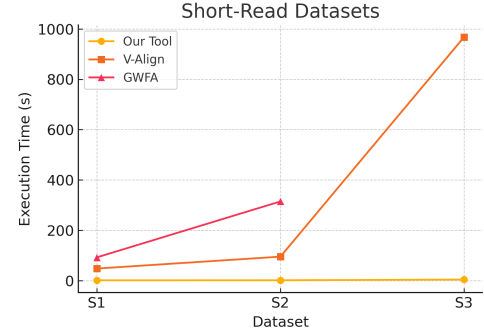


Fig. 3. Execution time comparison on short-read datasets (S1-S3).

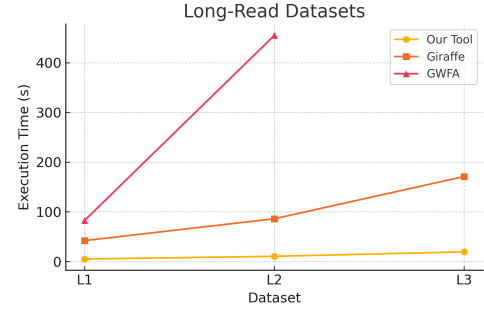


Fig. 4. Execution time comparison on long-read datasets (L1-L3).

on S2. On S3, both competing tools take several minutes or timeout, while our aligner completes in just over 4 seconds, achieving a speedup of over 220× compared to V-Align.

The gap widens even further on long-read datasets (L1-L3). Competing tools either timeout (as in the case of V-Align and GWFA) or, like Giraffe, require 8× to 9× more time than our implementation.

The performance gap can be attributed to the near-optimal utilization of the GPU, achieved by balancing intra-task (within a single alignment) and inter-task (across multiple alignments) parallelism. This is combined with a dynamic programming kernel specifically optimized for partial order graphs. In addition, memory access patterns are designed for coalescence, while shared memory is used to reduce latency and improve throughput.

VI. CONCLUSION - FUTURE WORK

We presented a fully functional prototype of a sequence-to-graph aligner that leverages GPU architectures for high-throughput and optimal alignment computation. Unlike existing state-of-the-art tools such as PaSGAL—which performs multithreaded sequence-to-graph alignment on the CPU but is no longer maintained or functional—and HGA, which utilizes the GPU as a multi-core extension of the CPU without fully exploiting its parallel nature and which is also non-functional, our implementation is not only operational but architecturally optimized for GPU execution. By parallelizing both inter-sequence and intra-sequence alignment tasks, we achieve substantial performance gains while maintaining correctness.

Our implementation does not use heuristics and seeding strategies, computing the full dynamic programming matrix to ensure exact alignment scores. While this design choice may increase computational workload, it guarantees correctness and makes the aligner suitable as a baseline for evaluating heuristic methods. Despite its exhaustive nature, the implementation noticeably outperforms the state-of-the-art sequence-to-graph aligners evaluated in our tests—V-ALIGN, Giraffe, and GWFA.

Future work will focus on optimizations at both the host and device levels. On the host side, we plan to introduce CUDA streams to further parallelize data transfers between host and device, thereby reducing overall computation time. On the kernel side, we aim to explore classic heuristics such as banded alignment and pruning strategies to reduce potentially unnecessary computation, which may unlock further speedups without compromising accuracy when full optimality is not strictly required.

REFERENCES

- [1] F. Peverelli et al., "GPU accelerated partial order multiple sequence alignment for long reads self-correction," 2020 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), New Orleans, LA, USA, 2020, pp. 1-9.
- [2] F. Zonghao and L. Qiong, Accelerating Sequence-to-Graph Alignment on Heterogeneous Processors, 2021.
- [3] C. Jain, S. Misra, H. Zhang, A. Dilthey and S. Aluru, "Accelerating Sequence Alignment to Graphs," 2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS), Rio de Janeiro, Brazil, 2019, pp. 451-461.
- [4] V. N. S. Kavya, K. Tayal, R. Srinivasan, and N. Sivadasan, "Sequence Alignment on Directed Graphs," *Journal of Computational Biology*, vol. 26, no. 1, pp. 53-67, 2019.
- [5] H. Zhang, S. Wu, S. Aluru, and H. Li, "Fast sequence to graph alignment using the graph wavefront algorithm," *arXiv preprint arXiv:2206.13574*, 2022.
- [6] J. Sirén, J. Monlong, X. Chang, A. M. Novak, J. M. Eizenga, C. Markello, J. A. Sibbesen, G. Hickey, P.-C. Chang, A. Carroll et al., "Genotyping common, large structural variations in 5,202 genomes using pangenomes, the giraffe mapper, and the vg toolkit," *bioRxiv*, pp. 2020-12, 2020.
- [7] M. Coggi, G. W. Di Donato, and M. D. Santambrogio, "A novel methodology for a comprehensive analysis of genomic sequence-to-graph alignment tools," *IEEE Transactions on Computational Biology and Bioinformatics*, vol. 22, no. 1, pp. 228-239, Jan.-Feb. 2025.
- [8] Wang, Ting, et al. "The Human Pangenome Project: a global resource to map genomic diversity." *Nature* 604.7906 (2022): 437-446.